# How to write a plugin for nopCommerce

14-18 minutes

---

- Actualizado:30/09/2025
- 

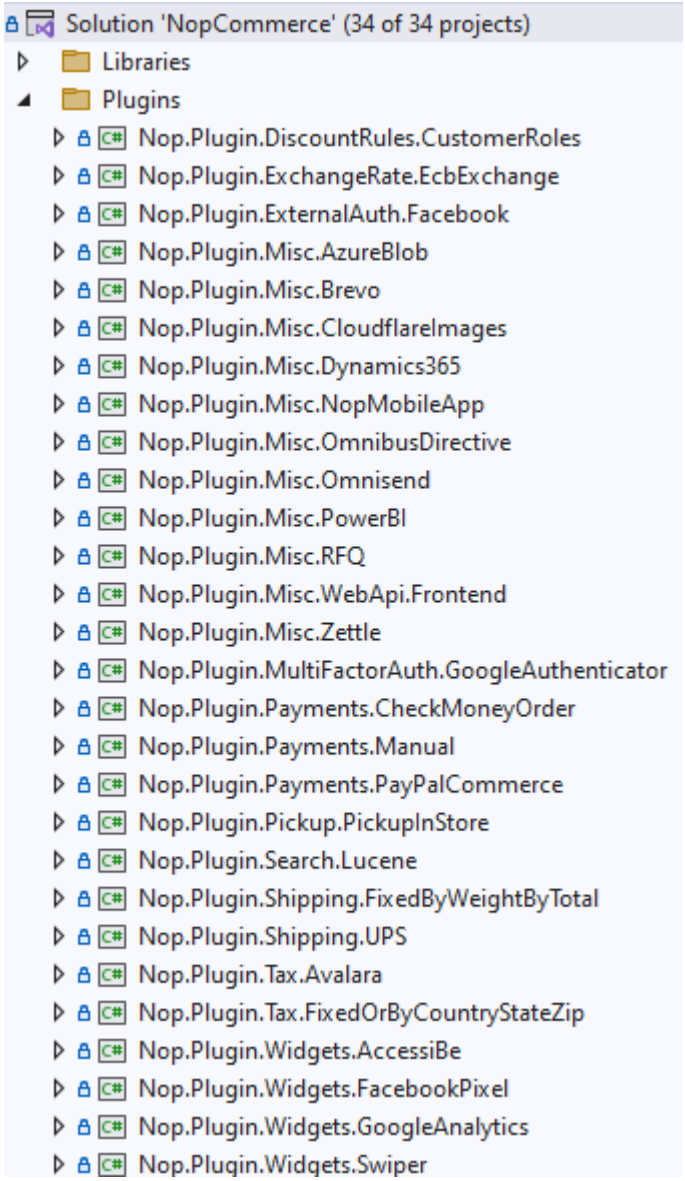
## Cómo escribir un plugin para nopCommerce

Los plugins se utilizan para ampliar la funcionalidad de nopCommerce. nopCommerce cuenta con varios tipos de plugins. Por ejemplo, métodos de pago (como PayPal), proveedores de impuestos, métodos de cálculo de envío (como UPS, USPS, FedEx), widgets (como el bloque de chat en vivo) y muchos más. nopCommerce ya incluye numerosos plugins. También puedes buscar plugins en el sitio web oficial de nopCommerce para ver si alguien ya ha creado uno que se ajuste a tus necesidades. Si no, este artículo te guiará en el proceso de creación de tu propio plugin.

## La estructura del plugin, los archivos necesarios y las ubicaciones.

1. Lo primero que debes hacer es crear un nuevo *`Class Library`*proyecto en la solución. Es recomendable colocar todos los complementos en el `\Plugins`directorio raíz de la solución (no los confundas con `\Plugins`los subdirectorios que se encuentran dentro del `\Nop.Web`directorio principal, que se utiliza para los complementos ya implementados). También es recomendable colocar todos los complementos en la `Plugins`carpeta de la solución.

   Un nombre recomendado para un proyecto de plugin es **`Nop.Plugin.{Group}.{Name}`**. **`{Group}`**es tu grupo de plugins (por ejemplo, *Pago* o *Envío* ). **`{Name}`**es el nombre de tu plugin (por ejemplo, *PayPalCommerce* ). Por ejemplo, el plugin de pago PayPal Commerce tiene el siguiente nombre: **`Nop.Plugin.Payments.PayPalCommerce`**. Pero ten en cuenta que no es un requisito. Y puedes elegir cualquier nombre para un plugin. Por ejemplo, `MyGreatPlugin`.




2. Una vez creado el proyecto del plugin, debes abrir su `.csproj`archivo en cualquier editor de texto y reemplazar su contenido con el siguiente:

```
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net9.0</TargetFramework>
        <Copyright>SOME_COPYRIGHT</Copyright>
        <Company>YOUR_COMPANY</Company>
        <Authors>SOME_AUTHORS</Authors>

<PackageLicenseUrl>PACKAGE_LICENSE_URL</PackageLicenseUrl>

<PackageProjectUrl>PACKAGE_PROJECT_URL</PackageProjectUrl>
        <RepositoryUrl>REPOSITORY_URL</RepositoryUrl>
        <RepositoryType>Git</RepositoryType>

<OutputPath>$(SolutionDir)\Presentation\Nop.Web\Plugins\PLUGIN_OUTP
        <OutDir>$(OutputPath)</OutDir>
        <!--Set this parameter to true to get the dlls copied
```

```
from the NuGet cache to the output of your    project. You need
to set this parameter to true if your plugin has a nuget package
to ensure that   the dlls copied from the NuGet cache to the
output of your project-->

<CopyLocalLockFileAssemblies>true</CopyLocalLockFileAssemblies>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference
Include="$(SolutionDir)\Presentation\Nop.Web.Framework\Nop.Web.Fram
/>
        <ClearPluginAssemblies
Include="$(SolutionDir)\Build\ClearPluginAssemblies.proj" />
    </ItemGroup>
    <!-- This target executes after "Build" target -->
    <Target Name="NopTarget" AfterTargets="Build">
        <!-- Delete unnecessary libraries from plugins path -->
        <MSBuild Projects="@(ClearPluginAssemblies)"
Properties="PluginPath=$(OutDir)" Targets="NopClear" />
    </Target>
</Project>
```

**Consejo**

Donde **PLUGIN_OUTPUT_DIRECTORY** debe reemplazarse con el nombre del complemento, por ejemplo, `Payments.PayPalCommerce`.

Lo hacemos de esta manera para poder usar un nuevo método para agregar referencias de terceros, introducido en .NET Core. Pero, en realidad, no es necesario. Además, las referencias de las bibliotecas ya referenciadas se cargarán automáticamente. Por lo tanto, es muy conveniente.

3. El siguiente paso es crear un `plugin.json`archivo necesario para cada plugin. Este archivo contiene metainformación que describe tu plugin. Simplemente copia este archivo de cualquier otro plugin existente y modifícalo según tus necesidades. Para obtener más información sobre el `plugin.json`archivo, consultaarchivo json del complemento

4. The last required step is to create a class that implements **`IPlugin`** interface (`Nop.Services.Plugins` namespace). nopCommerce has **`BasePlugin`** class which already implements some `IPlugin` methods and allows you to avoid source code duplication. nopCommerce also provides you with some specific interfaces derived from `IPlugin`. For example, we have the `IPaymentMethod` interface which is used for creating new payment method plugins. It contains some methods which are specific only for payment methods such as *`ProcessPaymentAsync()`* or

*GetAdditionalHandlingFeeAsync()*. Currently, nopCommerce has the following specific plugin interfaces:

- **IPaymentMethod**. These plugins are used for payment processing.
- **IShippingRateComputationMethod**. These plugins are used for retrieving accepted delivery methods and appropriate shipping rates. For example, UPS, UPS, FedEx, etc.
- **IPickupPointProvider**. These plugins are used for providing pickup points.
- **ITaxProvider**. Tax providers are used for getting tax rates.
- **IExchangeRateProvider**. Used for getting currency exchange rate.
- **IDiscountRequirementRule**. Allows you to create new discount rules such as "Billing country of a customer should be…"
- **IExternalAuthenticationMethod**. Used for creating external authentication methods such as Facebook, Twitter, OpenID, etc.
- **IMultiFactorAuthenticationMethod**. Used for creating multi-factor authentication methods such as *GoogleAuthenticator*, etc.

  **Note**

  This is a new interface, since version 4.40 we have provided the corresponding infrastructure for MFA integrations out of the box.

- **IWidgetPlugin**. It allows you to create widgets. Widgets are rendered on some parts of your site. For example, it can be a "Live chat" block on your site's left column.
- **IMiscPlugin**. If your plugin doesn't fit any of the interfaces above.

**Important**

After each project build, clean the solution before making changes. Some resources will be cached, which can lead to developer insanity.

You may need to rebuild your solution after adding your plugin. If you do not see DLLs for your plugin under `Nop.Web\Plugins\PLUGIN_OUTPUT_DIRECTORY`, you need to rebuild your solution. nopCommerce will not list your plugin in the *Local Plugins* page if your DLLs do not exist in the correct folder in `Nop.Web`.

# Handling requests. Controllers, models, and views

Now you can see the plugin by going to **Admin area** → **Configuration** → **Local Plugins**. But as you guessed our plugin does nothing. It does not even have a user interface for its configuration. Let's create a page to configure the plugin.

What we need to do now is create a controller, a model, and a view.

1. MVC controllers are responsible for responding to requests made against an ASP.NET Core MVC website. Each browser request is mapped to a particular controller.
2. A view contains the HTML markup and content that is sent to the browser. A view is the equivalent of a page when working with an `ASP.NET Core MVC` application.
3. An MVC model contains all of your application logic that is not contained in a view or a controller.

You can find more information about the MVC pattern here.

So let's start:

- **Create the model**. Add a **Models** folder in the new plugin, and then add a new model class that fits your needs.

- **Create the view**. Add a **Views** folder in the new plugin, and then add a `*.cshtml` file named `Configure.cshtml`. Set **"Build Action"** property of the view file is set to **"Content"**, and the **"Copy to Output Directory"** property is set to **"Copy always"**. Note that the configuration page should use `_ConfigurePlugin` layout.

- Also make sure that you have the `_ViewImports.cshtml` file in your \Views directory. You can just copy it from any other existing plugin.

- **Create the controller**. Add a **Controllers** folder in the new plugin, and then add a new controller class. A good practice is to name plugin controllers `{Group}{Name}Controller.cs`. For example, PaymentPayPalStandardController. Of course, it's not a requirement to name controllers this way (but just a recommendation). Then create an appropriate action method for the configuration page (in the admin area). Let's name it *`Configure`*. Prepare a model class and pass it to the following view using a physical view path: `~/Plugins/{PluginOutputDirectory}/Views/Configure.cshtml`.

- Use the following attributes for your action method:

```
[AutoValidateAntiforgeryToken]
[AuthorizeAdmin] //confirms access to the admin panel
[Area(AreaNames.ADMIN)] //specifies the area containing a
controller or action
public class PayPalCommerceController : BasePluginController
{
    public async Task<IActionResult> Configure(bool showtour =
false)
    {
        return
View("~/Plugins/Payments.PayPalCommerce/Views/Admin/Configure.cshtm
    }
}
```

**Tip**

You can also add these attributes directly to the controller. In this case, there is no need to tag each method with them.

For example, open `PayPalCommerce` payment plugin and look at its implementation of `PayPalCommerceController`.

Then for each plugin that has a configuration page, you should specify a configuration URL. Base class named `BasePlugin` has `GetConfigurationPageUrl` method which returns a configuration URL:

```
private readonly INopUrlHelper _nopUrlHelper;

public PayPalCommercePaymentMethod(INopUrlHelper nopUrlHelper)
{
    _nopUrlHelper = nopUrlHelper;
}

public override string GetConfigurationPageUrl()
{
    return
_nopUrlHelper.RouteUrl(PayPalCommerceDefaults.Route.Configuration);
}
```

*INopUrlHelper* is now used for routing outside of controllers. The names of the standard routes are stored in plugin defaults as constants.

```
/// <summary>
/// Represents the plugin constants
/// </summary>
public class PayPalCommerceDefaults
{
    /// <summary>
    /// Represents the route names
    /// </summary>
    public class Route
    {
        /// <summary>
        /// Gets the configuration route name
        /// </summary>
        public static string Configuration =>
"Plugin.Payments.PayPalCommerce.Configure";
    }
}
```

Registering the route to the plugin configuration page will look like below.

```
endpointRouteBuilder.MapControllerRoute(name:
PayPalCommerceDefaults.Route.Configuration,
    pattern: "Admin/PayPalCommerce/Configure",
    defaults: new { controller = "{CONTROLLER_NAME}", action = "
{ACTION_NAME}", area = AreaNames.ADMIN });
```

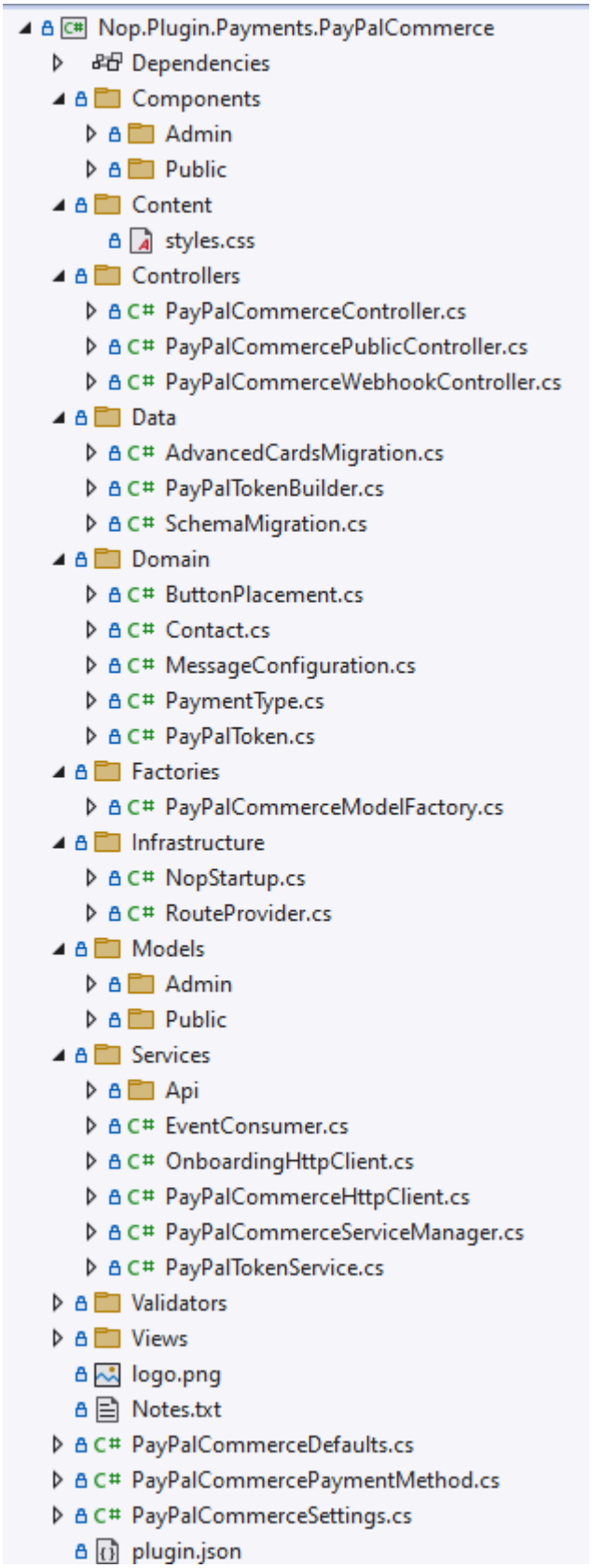
Where **{CONTROLLER_NAME}** is the name of your controller and **{ACTION_NAME}** is the name of the action (usually it's `Configure`).

Once you have installed your plugin and added the configuration method you will find a link to configure your plugin under **Admin** → **Configuration** → **Local Plugins**.

#### Tip

The easiest way to complete the steps described above is by opening any other plugin and copying these files into your plugin project. Then just rename appropriate classes and directories.

For example, the project structure of the *PayPalCommerce* plugin looks like the image below:




## Handling "InstallAsync", "UninstallAsync" and "UpdateAsync" methods

This step is optional. Some plugins can require additional logic during plugin installation. For example, a plugin can insert new locale resources. So open your `IPlugin` implementation (in most cases it'll be derived from the `BasePlugin` class) and override the following methods:

1. **InstallAsync**. This method will be invoked during plugin installation. You can initialize any settings here, insert new locale resources, or create some new database tables (if required).
2. **UninstallAsync**. This method will be invoked during plugin uninstallation.
3. **UpdateAsync**. This method will be invoked during the plugin update (when its version is changed in the `plugin.json` file).

#### Important

If you override one of these methods, do not hide its base implementation.

For example, the overridden `InstallAsync` method should include the following method call: *`base.Install()`*. The `InstallAsync` method of the *PayPalStandard* plugin looks like the code below

```
public override async Task InstallAsync()
{
    await _settingService.SaveSettingAsync(new
PayPalStandardPaymentSettings
    {
        UseSandbox = true
    });

    await _localizationService.AddOrUpdateLocaleResourceAsync(new
Dictionary<string, string>
    {
        ...
    });
    await base.InstallAsync();
}
```

#### Tip

The list of installed plugins is located in `\App_Data\plugins.json`. The list is created during installation.

## Routes

Here we will have a look at how to register plugin routes. ASP.NET Core routing is responsible for mapping incoming browser requests to particular MVC controller actions. You can find more routing information here. So follow the next steps:

If you need to add some custom route, then create the `RouteProvider.cs` file. It informs the nopCommerce system about plugin routes. For example, the following `RouteProvider` class adds a new route which can be accessed by opening your web browser and navigating to

`http://www.yourStore.com/Plugins/PayPalCommerceWebhook/WebhookHandler`
URL :

```
public class RouteProvider : IRouteProvider
    {
        /// <summary>
        /// Register routes
        /// </summary>
        /// <param name="endpointRouteBuilder">Route builder</param>
        public void RegisterRoutes(IEndpointRouteBuilder
endpointRouteBuilder)
        {
            ...
            endpointRouteBuilder.MapControllerRoute(name:
PayPalCommerceDefaults.Route.Webhook,
                pattern: "Plugins/PayPalCommerce/Webhook",
                defaults: new { controller = "PayPalCommerceWebhook",
action = "WebhookHandler" });
        }

        /// <summary>
        /// Gets a priority of route provider
        /// </summary>
        public int Priority => 0;
    }
```

## Upgrading nopCommerce may break plugins

Some plugins may become outdated and no longer work with the newer version of nopCommerce. If you have issues after upgrading to the newer version, delete the plugin and visit the official nopCommerce website to see if a newer version is available. Many plugin authors will upgrade their plugins to accommodate the newer version, however, some will not and their plugins will become obsolete with the improvements in nopCommerce. But in most cases, you can simply open an appropriate `plugin.json` file and update the **SupportedVersions** field.

## Conclusion

Hopefully, this will get you started with nopCommerce and prepare you to build more elaborate plugins.

## Plugin template

Puedes usar nuestra plantilla de Visual Studio para nuevos plugins de nopCommerce. Esto puede ahorrar mucho tiempo a los desarrolladores, ya que ahora no tienen que realizar manualmente todos los pasos iniciales, como la creación de carpetas (Controladores, Vistas, Modelos, etc.) y otros archivos necesarios (PluginNopStartup.cs,

_ViewImports.cshtml, ObjectContex, plugin.json, configuración, referencias del proyecto, etc.). Puedes encontrarla y las instrucciones de instalación aquí .